

CARTMAN AND THE GOLDEN CUBE

Hecho por Yura Hernández y Juan José Bolívar

Pontificia Universidad Javeriana Cali

Proyecto Final Arboles y Grafos

Profesor: Carlos Ramírez

Descripción general del problema:

El problema consiste en un cubo que rueda por una cuadrícula y que al caer sobre cada celda intercambia el oro que tiene en su cara inferior con el oro que pueda haber en esa celda. En el mapa hay exactamente seis pedazos de oro distribuidos y el objetivo es encontrar la ruta más económica para que el cubo los recoja todos simultáneamente, es decir, que los seis queden dentro del cubo al mismo tiempo. La dificultad radica en que, como el cubo es tridimensional y va rodando, se debe llevar un registro bastante fino sobre de qué cara está apuntando hacia abajo en cada momento, esto principalmente porque cada cara es independiente y puede o no contener oro. Además, cada movimiento tiene un costo que varía dependiendo de que, si en ese paso se recogió un nuevo pedazo de oro o no, lo que convierte el problema en una búsqueda del camino óptimo considerando simultáneamente la posición del cubo, su orientación y el estado de recolección del oro. Por esto es que nosotros al darnos cuenta de búsqueda de caminos óptimos, pensamos principalmente en un Dijkstra. (La forma en como llegamos a esa conclusión está especificada mejor en la bitácora)

Especificación del problema:

Entrada: Una matriz de $(RC) \in \{0 \dots R\} \times \{0 \dots C\}$ de caracteres f los cuales $f \in \{., S, G\}$ y con dos enteros $(A, B) \in \mathbb{N}$ con una función $c: A \cdot [f_i = .] + B \cdot [f_i = G]$.

Salida: La transformación de la matriz $(RC) \in \{0 \dots R\} \times \{0 \dots C\}$ a un Grafo $G = (V, E)$ no dirigido con una función de peso $w: E \rightarrow \mathbb{N}$

donde w se define como $v \in V$ que representa un estado del sistema y aristas $(u, v) \in E$.

$$w(G) = \sum c.$$

Definición de casos de prueba:

Vamos a decir que los estados se ven de la siguiente forma

$([0,0,0,0,0,0], (x,y), 0, 1, [(x,y), (x,y), (x,y), (x,y), (x,y), (x,y)])$

Donde la primera lista representa si cada cara tiene o no oro, la segunda la posición del dado, la tercera el costo acumulado, la cuarta es en que cara estamos (La parte inferior) siendo de 1 a 6 debido a que tenemos 6 caras y la última representa la posición de cada oro. La representación en los casos de prueba es una traza de ejecución para ilustrar el recorrido, no el estado formal.

Caso #1)

Tenemos la siguiente entrada:

```
4 4 1 2
S . . .
G . G .
G G . .
G . . G
```

El estado inicial seria:

```
(
[0,0,0,0,0,0],
(0,0),
0,
1,
[(1,0), (1,2), (2,0), (2,1), (3,0), (3,3)]
)
```

Donde el cubo está en la posición 0,0 ninguna cara tiene oro y tenemos un costo de 0.

Bajamos:

(
[0,0,0,0,1,0],
(1,0),
2,
5,
[(1,2), (2,0), (2,1), (3,0), (3,3)]
)

Ahora la cara 5 tiene oro y actualizamos el costo sumándole B ya que agarramos 1 de oro.

Bajamos:

(
[0,0,0,0,1,1],
(2,0),
4,
6,
[(1,2), (2,1), (3,0), (3,3)]
)

Volvemos a actualizar el costo sumándole B y marcamos la cara 6 con que tiene oro.

Bajamos:

(
[0,1,0,0,1,1],
(3,0),
6,
2,
[(1,2), (2,1), (3,3)]
)

Volvemos a actualizar el valor del costo sumándole B ya que agarramos oro y marcamos la cara 2 con oro.

Derecha:

(
[0,1,0,0,1,1],
(3,1),
7,
4,

[(1,2), (2,1), (3,3)]
)

Al costo se le suma A ya que no hay oro

Derecha:

(
[0,1,0,0,1,1],
(3,2),
8,
5,
[(1,2), (2,1), (3,3)]
)

Al costo se le suma A ya que no hay oro

Derecha:

(
[0,1,1,0,1,1],
(3,3),
10,
3,
[(1,2), (2,1)]
)

Al costo se le suma B ya que hay oro y se marca la cara 3.

Arriba:

(
[0,1,1,0,1,1],
(2,3),
11,
6,
[(1,2), (2,1)]
)

Como no hay oro entonces se le suma A al costo.

Izquierda:

(
[0,1,1,0,1,1],

(2,2),
11,
5,
[(1,2), (2,1)]
)

No hay oro entonces se le suma A al costo

Arriba:

(
[0,1,1,1,1,1],
(1,2),
13,
4,
[(2,1)]
)

Se encontró oro en la cara 4, entonces se actualiza el costo sumándole B y se marca la cara 4.

Abajo:

(
[0,1,1,1,1,1],
(2,2),
14,
5,
[(2,1)]
)

No se encontró oro entonces se le suma A al costo.

Izquierda:

(
[1,1,1,1,1,1],
(2,1),
16,
1,
[]
)

Se encuentra oro y se le suma B al costo y se marca la cara 1 con oro.

Esto termina el caso dándonos como salida:

Screw you guys, I got all the gold for 16 cost!

Caso #2)

4 4 1 2

S . . .

G . . .

G G G .

. . G G

El estado inicial sería el siguiente:

(
[0,0,0,0,0,0],
(0,0),
0,
1,
[(1,0), (2,0), (2,1), (2,2), (3,2), (3,3)]
)

Abajo:

(
[0,0,0,0,1,0],
(1,0),
2,
5,
[(2,0), (2,1), (2,2), (3,2), (3,3)]
)

Abajo:

(
[0,0,0,0,1,1],
(2,0),
4,
6,
[(2,1), (2,2), (3,2), (3,3)]
)

Derecha:

(
[0,0,0,1,1,1],
(2,1),
6,
4,
[(2,2), (3,2), (3,3)]
)

Derecha:

(
 [1,0,0,1,1,1],
 (2,2),
 8,
 1,
 [(3,2), (3,3)]
)

Abajo:

(
 [1,1,0,1,1,1],
 (3,2),
 10,
 2,
 [(3,3)]
)

Derecha:

(
 [1,1,1,1,1,1],
 (3,3),
 12,
 3,
 []
)

Aquí termina el caso y se tiene la siguiente salida:

Screw you guys, I got all the gold for 12 cost!

Caso #3)

5 5 2 4
 .. G ..
 .. G ..
 S G G G .
 G

El estado inicial sería el siguiente:

(
 [0,0,0,0,0,0],
 (2,0),
 0,
 1,
 [(0,2), (1,2), (2,1), (2,2), (2,3), (3,0)]
)

Abajo:

(
 [0,0,0,0,1,0],
 (3,0),
 4,
 5,
 [(0,2), (1,2), (2,1), (2,2), (2,3)]
)

Arriba:

(
 [0,0,0,0,1,0],
 (2,0),
 6,
 1,
 [(0,2), (1,2), (2,1), (2,2), (2,3)]
)

Derecha:

(
 [0,0,0,1,1,0],
 (2,1),
 10,
 4,
 [(0,2), (1,2), (2,2), (2,3)]
)

Derecha:

(
 [0,0,0,1,1,1],
 (2,2),
 14,
 6,
 [(0,2), (1,2), (2,3)]
)

Arriba:

(
 [0,1,0,1,1,1],
 (1,2),
 18,
 2,
 [(0,2), (2,3)]
)

Arriba

(
[1,1,0,1,1,1],
(0,2),
22,
1,
[(2,3)]
)

Abajo:

(
[1,1,0,1,1,1],
(1,2),
24,
2,
[(2,3)]
)

Abajo:

(
[1,1,0,1,1,1],
(2,2),
26,
6,
[(2,3)]
)

Derecha:

(
[1,1,1,1,1,1],
(2,3),
26,
3,
[]
)

Aquí termina el caso dando como salida:

Screw you guys, I got all the gold for 26 cost!

Ideas sobre la solución:

Tenemos la idea de hacer uso de Dijkstra ya que el grafo de estados planteado tiene pesos, a parte de eso la tupla debe de ser modificada en cuanto a las modificaciones de la posición

del oro cuando se sueltan, a parte de eso creemos que podemos implementar el marcado de caras por medio de mascara de bits y hacer un truco con el numero de cara para prender el bit cuando sea necesario.

Bitacora del Proyecto final:

Miércoles 29 de abril de 2026: El profesor Carlos Ramírez asigna el proyecto.

Jueves 30 de abril de 2026: Yura y Juan José deciden realizar el proyecto de manera conjunta.

Fin de semana de los días 1-3 de mayo: Antes de iniciar el proyecto decidimos la estrategia en la que vamos a afrontar este, en primera instancia pensamos en que Juan José afrontase principalmente la programación y yura la descripción general y especificación del problema, incluyendo los casos de prueba. Esto lo decidimos porque sentimos que esos son los puntos más débiles de cada uno, por lo que trabajar en lo que no conocemos tanto nos ayudaría a mejorar, aunque luego descartaríamos esa idea porque no queremos que uno conozca mucho del tema y otro poco ya que esto nos puede pasar una mala jugada en la sustentación del proyecto.

Miércoles 6 de mayo de 2026: Definimos por completo la descripción general del problema, en primera instancia lo primero que notamos es que este programa tiene similitudes con problemas subidos anteriormente en la arena, específicamente los problemas Together (en la tarea 5 del curso de arboles y grafos 2026-1) y die, (en la tarea 3 del curso de estructuras de datos de 2025-2). Las similitudes que encontramos respectivamente son, grafos de estado y la orientación del dado o cubo al rotarlo en las posibles direcciones a las cuales nos podemos mover. Juan José no vio el curso de estructuras de datos con Carlos, pero si lo vio

con Gerardo, por lo que, si realizó la tarea y recordaba usar un struct para el dado, por lo que un primer acercamiento al código podría ser ese. Por otro lado, Yura denotó que al ser un grafo de estado lo primero que se le llegó a la cabeza fue hacer un bfs o un Dijkstra a lo cual Juan José estuvo de acuerdo, por lo que como primer acercamiento al código tenemos esos pequeños incisos.

Sábado 9 de mayo de 2026: Realizamos la especificación del problema, decidimos realizarlo en forma de tuplas ya que es la forma mas efectiva y entendible en la que tu puedes definir un grafo de estados, esto ya lo teníamos entendido ya que habíamos estudiado con anterioridad el problema de sokoban, el cual luego apareció en el parcial, para este teníamos que definir de la misma manera este grafo de estados, por lo que teníamos en mente lo que teníamos que realizar. Juan José no tenía muy claro porque las posiciones del oro se guardaban en un vector de máximo 6 posiciones, él pensaba que se tenía que guardar constantemente el mapa completo para poder visualizar las posiciones del oro, pero yura le explicó que esto además de ser poco eficiente ya que estaríamos guardando el mapa completo, esto directamente no es correcto, ya que el mapa no se considera como un estado como tal, ya que aunque las posiciones en las que está el oro pueden cambiar, el mapa como tal NO cambia, por lo que no se considera como un estado.

Martes 12 de mayo de 2026: En clase el profe especificó que NO podíamos mencionar dentro de la especificación del problema cosas como tuplas, ya que eso de cierta manera ya infiere una forma de solucionar el problema, y como la especificación es general, nuestro acercamiento puede llegar a un camino específico, el cual no termine cubriendo todos los casos generales, por lo

que nos tocó volver a modificar y realizar de nuevo la especificación del problema.

Jueves 14 de mayo de 2026: Yura y Juan Jose deciden hacer un GitHub para subir los documentos e irlos actualizando automáticamente.

Viernes 15 de mayo de 2026: Al hacer los casos de prueba nos dimos cuenta de que necesitamos guardar la posición de las piezas de oro ya que necesitamos saber en donde están en cada instante, de igual manera usar una lista con las posiciones iniciales en el estado puede que no sea una buena opción, es algo que tenemos que pensar de mejor manera. De igual manera es un buen primer acercamiento al método de estados finales que vamos a usar.

Sábado 16 de mayo: Antes de entregar el documento Yura y Juan José se reunieron por última vez para revisar y terminar de chequear las especificaciones del problema, encontramos unas inconsistencias entre lo que decíamos y luego lo que analizábamos en los casos de prueba. Fuimos muy rigurosos siendo conscientes de que las especificaciones no tienen que basarse en un caso de prueba específico (como el que nosotros planteamos) si no que tienen que basarse en hacer una cobertura correcta de todas las posibilidades del problema, esto porque nada nos asegura que los casos de prueba finales de este programa sean iguales a los que usamos nosotros para hacer nuestro análisis.

